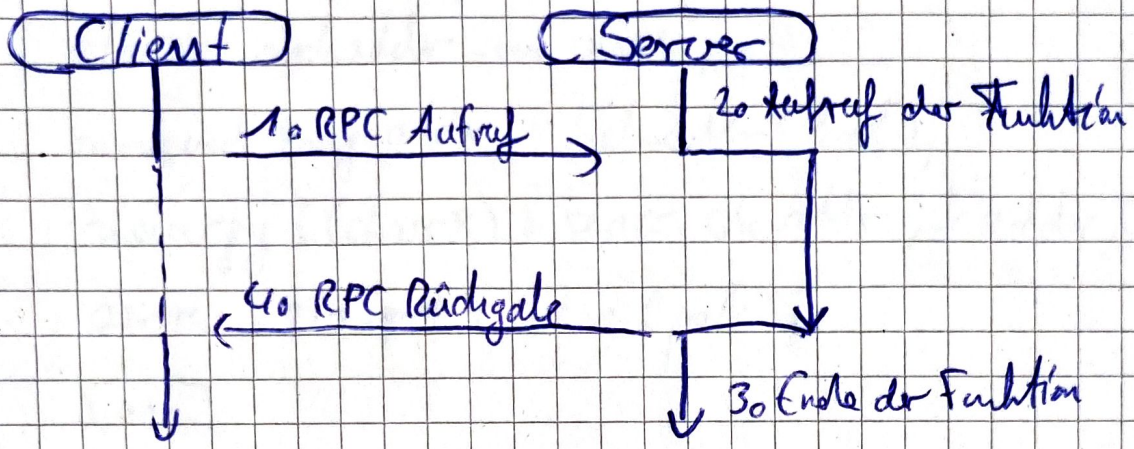
VS - KVB 2

- Entfernter Aufruf (Remote Procedure Call, RPC)

Entfernter Aufruf ist eine Technik zur Realisierung von Interprozesskommunikation. Sie ermöglicht den Aufruf von Funktionen in anderen Adressräumen.



→ Unterschiede zu lokalen Aufrufen:
- ↳ Verschiedene Rechner bedeutet verschiedene Adressräume
- ↳ Rechner verwenden unterschiedliche Datenstrukturen
- ↳ Aufruf und Rückgabe erfolgen über ein Kommunikationsnetz
- ↳ Fehlersituationen

→ Gegenüber der Nutzung einer lokalen Funktion sollte
- ↳ der zusätzliche Ausführungsaufwand minimal sein
- ↳ Softwaretechnisch sollten sich Aufrufe nicht unterscheiden.

→ Entfernte Aufrufe sind Nachbildung lokaler Aufrufe

→ Nicht alle Aspekte können transparent gemacht werden :(

VS - HVB

→ Umsetzung von Entfernten Aufrufen:

Entfernte Funktion: double kleinsterEigenwert(double[][] m, int n)

Lokale Funktion Client Stub:

```
double kleinsterEigenwert(double[][] m, int n){
    struct sockaddr_in server;
    unsigned long addr = inet_addr(host);
    memcpy((char*) &server.sin_addr, &addr, [...])
    server.sin_port = htons(port);
    [...]
    if(connect(socket, (struct sockaddr*) &server, [...]) < 0){
    }
    //sende m & n
    //empfange Anzahl
    reelle Eigenw
}
```

→ Übertragung der Parameter:

→ TCP leistet nur eine Übertragung eines Byte-Stroms.

→ Alle Daten müssen vor der Übertragung in Byte-Folge umgewandelt werden.

VS - KVB

- **Externe Datenrepräsentation**
  Datenformat, welches bei der Übertragung von Daten in einem Kommunikationssystem verwendet wird.

- **Marshalling, Unmarshalling**
  Die Umwandlung von Daten im lokalen Datenformat in das externe Format nennt man Marshalling, den umgekehrten Vorgang Unmarshalling. Häufig werden auch die Begriffe serialisieren und deserialisieren verwendet

↝ CDR = Common Data Representation
- → Datentypen: 15 elementare Typen, 6 zusammengesetzte Typen
- → Darstellung: → Bytefolge
  - → Elementare Typen: ist n die Anzahl der Bytes, dann Ablage an Index position, welche Vielfaches von n ist.
- → Jeder Wert hat einen eindeutigen Typ, Empfänger muss diesen Typ kennen → Unterschied zu JSON

→ JSON vs. CDR: → CDR benötigt zum Unmarshalling den Datentyp
- → CDR Marshalling ist kompakter als JSON

VS-KVB

- Remote Procedure Call (RPC)

  Aufruf von prozeduralen Funktionen in anderen Adressräumen: Client-Programme rufen Funktionen in Server-Programmen auf.

- Remote Method Invocation (RMI)

  Aufruf von objekt-orientierten Methoden in anderen Adressräumen: Objekte rufen Methoden anderer Objekte in einem anderen Prozess / Rechner auf.

→ Beschreibung der angebotenen Funktionen / Methoden über Schnittstellenbeschreibung

→ Trennung von Schnittstellen und Implementierung
  → Information Hiding
  → Abhängigkeit von Komponenten minimieren
  → Implementierung kann ohne Auswirkung auf andere Komponente ausgetauscht werden

→ Schnittstellen in VS:
  → Nur Methoden / Prozeduren daher direkter Variablenzugriff
  → Problem: Zeiger innerhalb eines Prozesses sind in anderen Prozessen ungültig.

→ homogene Systeme:
  C header files, C++ Klassen, Java Interfaces

VS-KVB

- Eine Schnittstellenbeschreibungsprache (IDL)

Eine Schnittstellenbeschreibungssprache oder Interface Definition Language (IDL) ist eine deklarative formale Sprache und beinhaltet eine Sprachsyntax zur Beschreibung von Schnittstellen einer Software-Komponente.

→ Mit IDLs lassen sich Funktionen mitsamt Parametern und Datentypen beschreiben, ohne Eigenschaften einer bestimmten Programmiersprache zu verwenden.

→ IDLs sind Erweiterungen von Externen Datenrepräsentationen

eg: CORBA IDL

```
struct Person {
    string name;
    string place;
    long year;
};
```

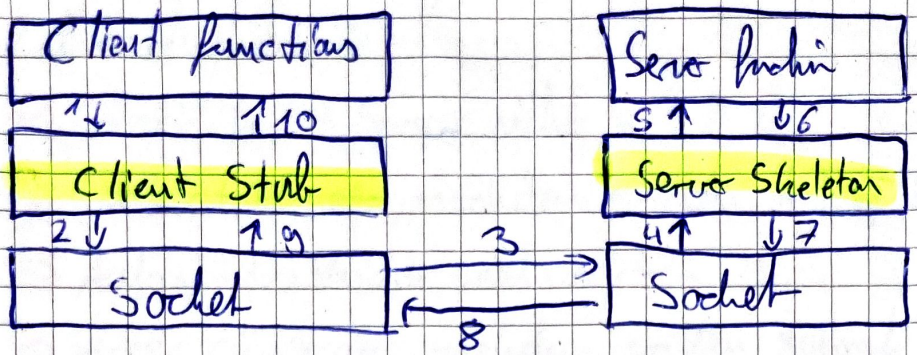
```
interface Person {
    readonly attribute string list name;
    void add Person (in Person p);
    void del Person (in string name, out Person p);
};
```

→ Umsetzung von RPCs

→ Ablauf:
- → Werkzeug generiert Code für entfernten Aufruf
- → Die generierten Codeteile nennt man stubs oder skeletons
- → Stubs sind lokale Prozeduren, die aus der eigentlichen Prozedur von einem Werkzeug generiert werden
- → Middleware unterstützt Ausführung des generierten Codes

→ Welche Informationen benötigt ein solches Werkzeug?
- → Name der Funktionen
- → Typen von Parametern und Rückgabewert
- → Adresse des Servers

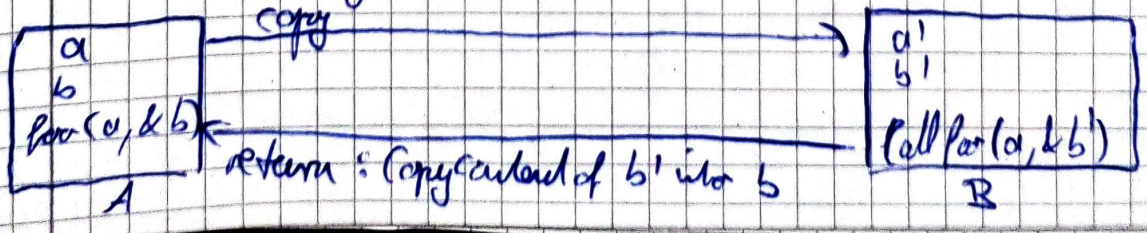
VS-KVB



→ Client Stub:
- ↳ ist ein Proxy, d.h. gleiche Schnittstelle wie entfernte Prozedur
- ↳ steuert Übertragungsprotokoll
- ↳ marshallt und verpackt Parameter in Nachricht
- ↳ verschickt Nachricht, wartet auf Antwort und packt diese aus.

→ Server Skeleton:
- ↳ wartet auf Anfragen
- ↳ packt Parameter aus, ruft lokale Prozedur auf, packt Rückgabewert in Nachricht und verschickt die

→ RPC: Behandlung von Zeiger-Typen
- → Call-By-Reference kann bei RPCs nicht angewendet werden, da Adressen sich auf verschiedene Adressräume beziehen
- → Lösung: call-by-copy/restore
- ↳ Bei Aufruf: Übertragung wie bei Call-By-Value
- ↳ Nach Beendigung: Zurückschicken zum Client und Überschreiben der alten Werte



VS - KVB

→ Fehlersituationen:

- → Client findet Server nicht
- → Client stürzt ab, nachdem er den Aufruf abgesendet hat
- → Aufrag-Nachricht geht verloren
- → Server stürzt ab, nachdem er den Aufruf empfangen hat,
  - ▶ aber bevor er die Antwort abgesendet hat
  - ▶ aber bevor er den Aufrag ausgeführt hat
- → Antwort geht verloren
- → Client stürzt ab, bevor er Antwort erhält.

→ Verlorener Aufrag-Nachricht:

→ Übliche Gegenmaßnahmen mit Timeout und Wiederholung

→ Probleme:

- → Max Anzahl der Wiederholungsversuche?
- → Wie groß ist Timeout?
- → Aufrag-Nachricht nicht verloren, sondern untypisches Verhalten vom Server → Doppelte Aufrag-MSG
  ↔ Server sollte duplikate erkennen

VS - UVS

→ Verlorene Antwort - Nachricht:

1) Analog zu verlorener Aufruf-MSG
   - Das Client Sicht nicht unterscheidbar
   - Es muss verhindert werden, dass Funktionen mit Seiteneffekt ungewollt mehrfach ausgeführt werden.

2) Server hält Historie versendeter Antworten
   - Falls Server Aufruf-Duplikat erkennt und Aufruf bereits ausgeführt hat
     - ↳ letzte Antwort erneut senden, ohne Funktion auszuführen
   - Der Client muss nur die letzte Antwort gespeichert werden.

- Aufrufsemantik

Die Aufrufsemantik eines RPC beschreibt, welche Garantien dem Client bezüglich der Ausführung einer Prozedur bei bestimmten Fehlerfällen gemacht werden.

→ Mindestens Einmal
→ Höchstens Einmal
→ Genau Einmal :)

VS - KB

~> Entfernter Aufruf in OO-Sprachen:

-> Grundprinzip von RPC für OO-Sprachen identisch, jedoch sind Erweiterungen notwendig:

-> Methode wird im Kontext von Objekten ausgeführt

-> Objekte als Parameter

-> Vererbung

-> Polymorphismus

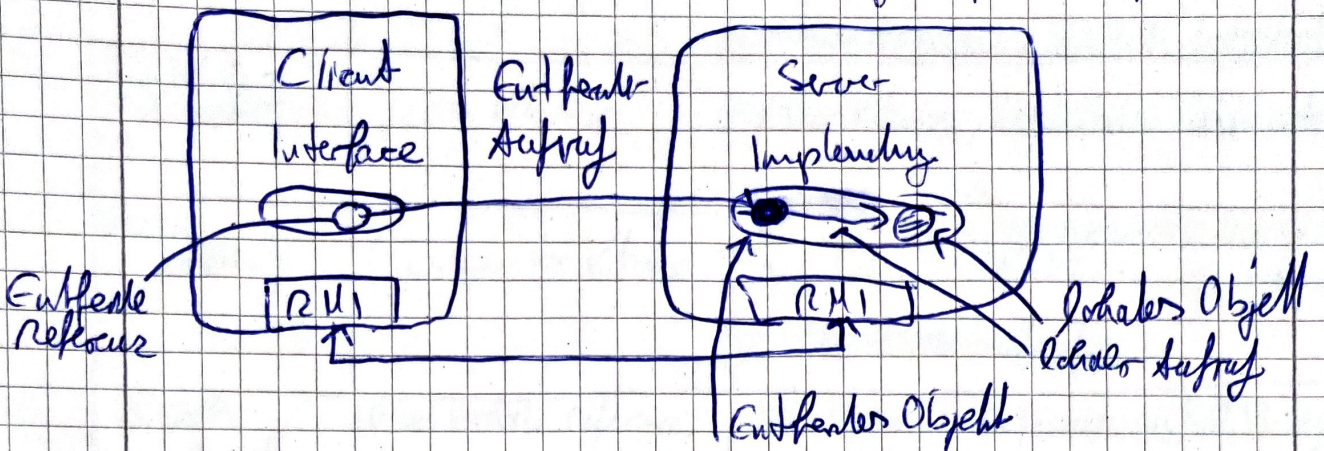
-> Um Methode eines entfernten Objektes aufzurufen muss Referenz auf dieses Objekt existieren

Nicht-verteiltes Objektmodell	Verteiltes Objektmodell
• Objekt Referenzen	• Entfernte Methodenaufrufe:
• Methodenaufrufe haben Objektzustand ändern	↳ aufrufendes Objekt und empfangenes Objekt auf verschiedenen Rechnern
• Klassen	• Lokale Methodenaufrufe
• Ausnahmen	↳ aufrufendes Objekt und empfangenes Objekt auf einem Rechner
• Freispeicherverwaltung	• Schnittstellen
	• Objekte in anderem Prozess/Rechner heißen entfernte Objekte
	• Entfernte Referenzen (Proxy)

VS-II VB

→ RMI: Remote Method Invocation

→ Entfernter Methodenaufruf bezieht entfernte Referenzen



→ Zusammenspiel: Client, Server & Registry

→ Server stellt entfernte Objekte mit Implementierung bereit

↳ Methoden werden auf Server ausgeführt

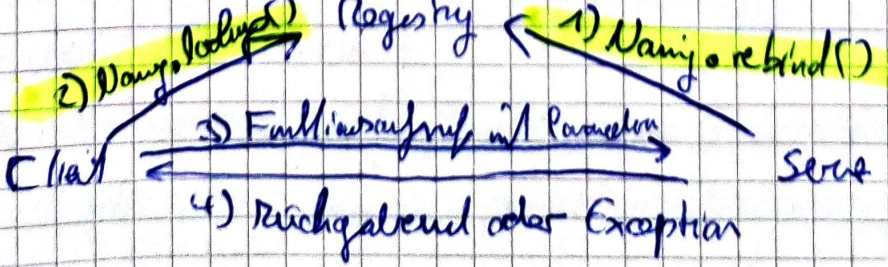
↳ Server-Stub leitet eingehende Aufrufe vom Netzwerk an referenziertes Objekt weiter und führt Methode aus

→ Namensdienst (Registry):

↳ Dienst, der Objekte mit einem eindeutigen Namen verbindet

↳ Server registriert Objekt bei Namensdienst

→ Ablauf:
- Client erfragt entfernte Referenz beim Namensdienst und ruft Methode auf
- Methoden kann entfernte Referenzen zurückgeben
- Deshalb müssen nicht alle Objekte in Registry registriert werden.



VS - KVO

	Lokales Objekt	Entferntes Objekt
Objektdefinition	Java Klasse	Entfernt aufrufbare Methoden sind in Interface, welches Remote erweitert, festgelegt
Objekt implementiert	Java Klasse	Java Klasse, welche das entfernte Interface implementiert
Objekt Erzeugung	Konstruktor der Klasse	Konstruktor der Klasse wird auf Server ausgeführt, Clients können diese Konstruktoren nicht direkt aufrufen
Objekt Zugriff	über lokale Referenz auf Objekt	über lokale Referenz auf Proxy des entfernten Objektes
Ausnahmen	Exception	Remote Exception

→ Wichtige Klassen und Schnittstellen:

- java.rmi.Remote
  - ↳ Schnittstellen, die alle entfernten Objekte implementieren müssen
- java.rmi.RemoteException
  - ↳ Unterklasse von IOException, entfernte Methoden können diese Exception werfen
- java.rmi.server.UnicastRemoteObject
  - ↳ Klassen von entfernten Objekten können Unterklasse dieser Klasse sein
- java.rmi.RMISecurityManager
  - ↳ Sicherheitsmanager bestimmt die Rechte von geladenen Klassen
- java.rmi.Naming
  - ↳ Klasse für den Zugriff auf den Namensserver
- java.io.Serializable
  - ↳ Parameter & Rückgabewerte müssen diese Schnittstellen implementieren, um sie per Kopie zu übergeben

VS - KVB

⇝ Entfernte Schnittstelle:

- → Interface Remote ist Erweiterung von Serializable
- → Alle Methoden listen RemoteException in throws-Klausel
- → Einschränkungen für Typen von Parametern/Rückgabewerten entfernter Methoden:

1) Primitive Werte und Objekte, deren Klasse Serializable implementiert, können als Parameter/Rückgabewert verwendet werden; Übertragung als Werte!
   - ▶ Wird solcher Objekt als Wert übergeben, so wird im Prozess des Empfängers ein neues Objekt erzeugt
   - ▶ Dieses Objekt ist unabhängig von Ursprungsobjekt, d.h. Änderungen an einem der Objekte haben keinen Einfluss auf den Zustand des anderen

2) Falls Typ eines Parameters/Rückgabewertes eine entfernte Schnittstelle ist, dann Übergabe als entfernter Objektreferenz

e.g.:

```
public interface Address extends Remote {
    public String getName() throws RemoteException;
    [...]
}
```

VS - KVB

~> Entfernte Klassen
- ↳ müssen entferntes Interface implementieren
- ↳ Bevor Clients entfernte Methode aufrufen können, müssen entfernte Objekte vom Server exportiert werden

↳ 2 Möglichkeiten:
1) Entfernte Klasse ist Unterklasse von UnicastRemoteObject
   ↳ solche Objekte werden bei Erzeugung automatisch exportiert
2) Server exportiert Objekte explizit mit Aufruf von exportObject

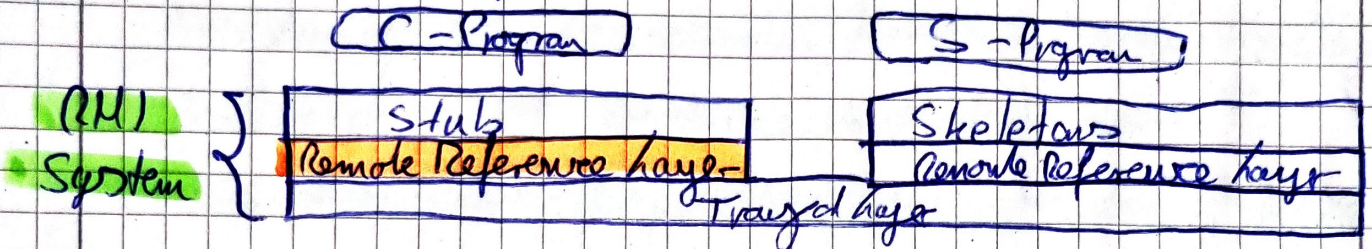
eg:

```
public class AdressListImpl extends UnicastRemoteObject implements AdressList {
    public Address createAddress() throws RemoteException {
        [...]
        return (Address) UnicastRemoteObject.exportObject(addr);
    }
    public void deleteAdress(Address addr) throws RemoteException {
        [...]
        UnicastRemoteObject.unexportObject(addr.getReg());
    }
}
```

VS-RVB

→ RMI

- RMI - Software
  - → Softwaremodul zwischen Objekten auf Appebene und Kommunikations- und entfernten Referenzmodul
  - → Stubs, Dispatcher, Skeletons



- RMI Reference Layer
  - → Übersetzung entfernte / lokale Objektreferenzen auf Basis von entfernter Objekt-Tabelle
  - → Erstellung entfernter Referenzen
- Stub
  - → Stellvertreterobjekt auf Client-Seite
  - → Macht entfernte Methodenaufrufe für Clients transparent
  - → Marshalling der Argumente, Unmarshalling des Rückgabewertes, senden und empfangen
  - → Verhält sich wie lokales Objekt
  - → Aufrufe werden nicht ausgeführt sondern Weiterleitung an entferntes Objekt
  - → Pro entferntes Objekt ein Stub

VS - KVS

→ RMI

- Skeleton
  - → Stellvertreterobjekt auf Server-Seite
  - → Implementiert entfernte Schnittstelle
  - → Aussparchen des Aufrufs
  - → Aufruf der entsprechenden Methode des entfernten Objektes
  - → Verpacken des Ergebnisses in der Antwort
  - → Pro entfernter Objektklasse eine Skeleton-Klasse
- Stubs & Skeletons werden vom Compiler rmic kompiliert erzeugt.
- Server-Programm enthält
  - → Initialisierungsteil, es wird mindestens ein entferntes Objekt generiert.
  - → Startpunkt für weitere Kommunikation
  - → Registrierung des Objektes in Registry
    - ↳ In Adresslistenverwaltung muss nur die Liste registriert werden.

→ Registry:

- → Registry ist eigenständiger Dienst, der auf zwei Arten gestartet werden kann:
  - vom Server selbst, mit LocateRegistry.createRegistry()
  - über externes Dienstprogramm rmiregistry
- → Verwaltung einer Tabelle mit entfernten Objekten auf diesem PC
- → Adressen: rmi://hostname:port/objektname
  - Defaultport: 1099
  - Defaulthost: localhost
  - Defaultprotokol: rmi

VS-KVB

reg:

⇒ Implementierung des Servers

```
public class AddressListRmiServer {
    [...]
    public void start() {
        Registry registry = null;
        try {
            System.setProperty("java.rmi.server.hostname", host);
            registry = LocateRegistry.createRegistry();
            registry.rebind(serviceName, addressList);
            [...]
    }
```

~> RMI Registry

-> Zugriff auf Registry über Klasse Naming

-> Statische Methoden:

- public static void bind(String name, Remote obj);
- public static void rebind(String name, Remote obj);
- public static Remote lookup(String name);
- public static void unbind(String name);
- public static String[] list(String name);

-> Alle werfen:

RemoteException, AccessException, NotBoundException, MalformedURLException

VS: KVB

→ eg:

→ Implementierung des Client

```
public class AddressClient {
    public static void main (String args[]) {
        Config cfg = new Config()
        cfg.parse(args);
        try {
            String key = "rmi://" + cfg.getHostname() + ":"
                    + cfg.getPort() + "/" + cfg.getServiceName();
            AdressList al = (AdressList) Naming.lookup(key)
            [...]
        }
    }
```

VS - KVS - Ex 4

(4-1)

1) Entfernter Aufruf?

Entfernter Aufruf:

Entfernter Aufruf ist eine Technik zur Realisierung von Interprozesskommunikation. Sie ermöglicht den Aufruf von Funktionen in anderen Adressräumen.

Remote Procedure Call (RPC):

Aufruf von prozeduralen Funktionen in anderen Adressräumen: Client-Programm ruft Funktionen im Server-Programm auf.

Remote Method Invocation (RMI):

Aufruf von OO-Methoden in anderen Adressräumen: Objekte rufen Methoden anderer Objekte in einem anderen Prozess / Rechner auf.

2) externe Datenrepräsentation?

Externe Datenrepräsentation:

Datenformat, welches bei der Übertragung von Daten in einem Kommunikationsnetz verwendet wird.

Marshalling, Unmarshalling:

Die Umwandlung von Daten in lokalen Datenformat in das externe Datenformat nennt man Marshalling, den umgekehrten Vorgang nennt man Unmarshalling

VS - KMB - Ex 4

(4-1)

a) Was sind Stubs?

Stubs sind lokale Prozeduren, die aus der eigentlichen Prozedur von einem Werkzeug generiert werden.

Client Stub: → ist ein Proxy
→ Steuert Übertragungsprotokoll
→ marshallt und verpackt Parameter in MSG
→ verschickt MSG, wartet auf Antwort, packt aus

Server - Stub (Skeleton): → wartet auf Anfragen
→ packt parameter aus, ruft lokale Parameter auf, packt Rückgabewert in MSG und sendet

b) Fehler bei entfernten Verfahren?
→ Client findet Server nicht
→ Client stürzt ab, nachdem er den Verfahrensaufruf abgeschickt hat
→ Verfahrens-MSG geht verloren
→ Server stürzt ab, nachdem er den Verfahrensaufruf empfangen hat,
→ aber bevor er die Antwort abgeschickt hat
→ aber bevor er den Verfahrensaufruf ausgeführt hat
→ Antwort geht verloren
→ Client stürzt ab bevor er Antwort erhält.

VS - KVD - Ex 4

(4-D)

e) Welche Kardinalität?

→ Höchstens - Einmal

→ Mindestens - Einmal

→ Genau einmal ✓

f) IDL?

Interface Definition Language (IDL):

Eine IDL ist eine deklarative formale Sprache und definiert Sprachsyntax zur Beschreibung von Schnittstellen einer Software-Komponente.

↳ generieren Stubs

VS - KVB - Ex 4

(4-2) RPC vs RMI

RPC in Client-Stub & Server-Stub
TCP/IP & synchrone Aufrufe.

- Welche Informationen muss der Client mit einem entfernten Funktionsaufruf mindestens an den Server senden?
  → Funktionsname, Parameter

- Was passiert, wenn der entfernte Funktionsaufruf im Stub auf Serverseite empfangen wird?
  → Stub packt Variable aus, ruft lokale Funktion auf, packt Rückgabewert in Variable und verschickt diese

- Warum ergibt es kein Sinn, einen Zeiger zu übergeben?
  → Andere Adressräume

- Welche Information wird ausschließlich vom Server zurück gesendet?
  → Rückgabewert
  → Ja / Nein bei Void Funktionen, Response macht Sinn damit Client weiß ob erfolgreich ausgeführt wurde
  → Exceptions?

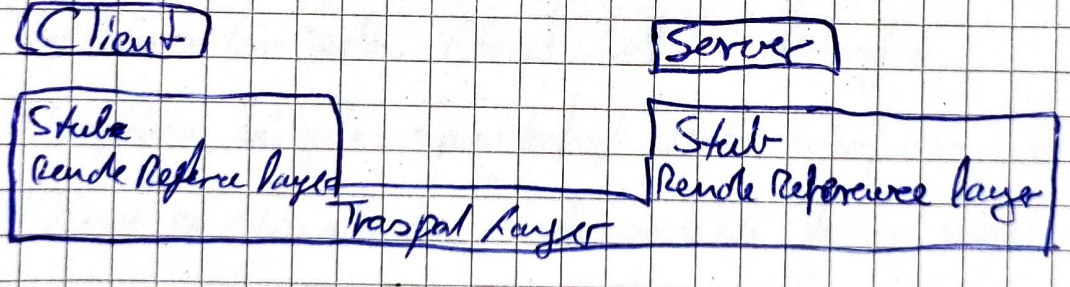
VS - KVS - 6a

(4-2)

- Wie äußert sich der Unterschied zwischen RPC & RMI in den Informationen, die vom Client bei einem entfernten Methodenaufruf an den Server gesendet werden?
  - → Von welchem Objekt wird Methode aufgerufen
  - → Objekte haben Parameter sein.

- Wie wird die Übergabe von Objekten als Referenz bei entfernten Methodenaufrufen umgesetzt?

→ Falls Typ eines Parameters eine entfernte Schnittstelle ist, dann Übergabe als entfernte Objektreferenz.
↳ frage Registry



Remote Reference Layer:
- → Übersetzung entfernte / lokale Objektreferenzen auf Basis von entfernter Objekt - ID Tabelle
- → Erstellung entfernter Referenzen

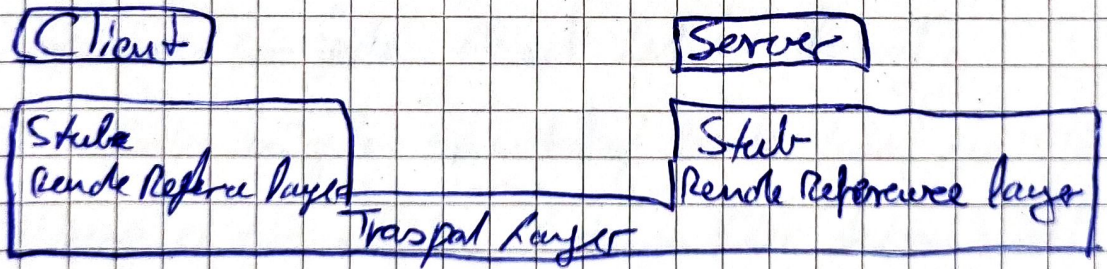
VS - KVS - 6a

(4-2)

- Wie äußert sich der Unterschied zwischen RPC & RMI in den Informationen, die vom Client bei einem entfernten Methodenaufruf an den Server gesendet werden?
  - → Von welchem Objekt wird Methode aufgerufen
  - → Objekte haben Parameter sein.
- Wie wird die Übergabe von Objekten als Referenz bei entfernten Methodenaufrufen umgesetzt?

→ Falls Typ eines Parameters eine entfernte Schnittstelle ist, dann Übergabe als entfernte Objektreferenz.

↳ Frage Registry



Remote Reference Layer:

- → Übersetzung entfernte / lokale Objektreferenzen auf Basis von entfernter Objekt - ID Tabelle
- → Erstellung entfernter Referenzen

VS-KVB-Ex 4

(4-2)

- Es sei A eine Java Klasse, welche Serializable aber nicht Remote implementiert. Eine Methode m einer entfernten Schnittstelle hat den Rückgabetyp A. Welches Objekt bekommt ein Client bei Aufruf von m zurückgegeben?

  → Wird A auf diesem Weg zurückgegeben so wird ein Proxy des Clients ein neues Objekt erzeugt.

  ↳ Dieses Objekt ist unabhängig vom Ursprungsobjekt

(4-3) Semantik von RPC

Protokoll mit Hochstens-Einmal-Semantik?

→ Speichere für jeden Client letzten Aufruf.

→ Vergleiche bei jedem neuen Aufruf letzten Aufruf des Clients

→ Wenn es der gleiche ist dann führe Aufruf nicht aus

VS-KVB-Ex 4

(4-1) Basic Encoding Rules (BER)

BER Standard für die Kodierung von Daten- & Typstrukturen.
(see Exercise)

Type	Length	Value

8 Bit
↓
2 Bit → ordnen dem Datentyp eine Sichtbarkeit/Klasse zu (hier 00 ≙ Universal)
1 Bit → (0 → Primitive) (1 → Komplex)
5 Bit → Datentyp

Länge: Kurz: [0|Byte] → # Byte in Value Feld
Lang: [1|7 Bit] → gibt Länge des folgenden Integer-Werts in Bytes an

a) 02h 01h E6h

0000 0010 | 0000 0001 | 1110 0110
Integer | 1 Byte | $= -2^7 + 2^6 + 2^5 + 2^2 + 2^1 = -26$

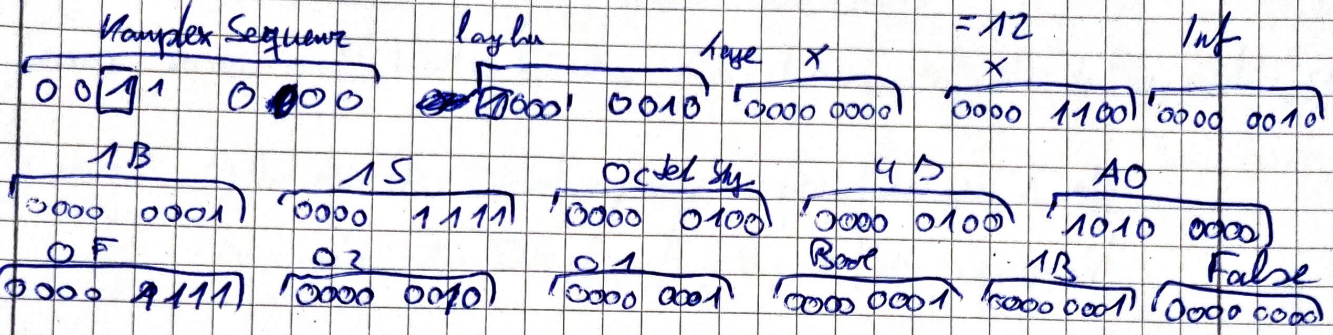
b) 01h 01h FFh
Bool | 1 Byte | true

c) 02h 01h 00h → Int = 0

VS-KVB-6a

(4-1)

a) 30h 82h 00h 0Ch 02h 01h 0Fh 04h 04h A0h
0Fh 02h 01h 01h 01h 00h



c) 20 Integer
02h 01h 14h

f) -80 Integer
02h 01h B0h

g) true
01h 01h FFh

g) '0x01 0x23 0x45'
04h 03h 01h 23h 45h

i) Sequenz (32 Int, true, '0x61 0x01'
30h 01h 0Ah 02h 01h 17h 01h 01h FFh
04h 02h 11h 0Ah)

Verteilte Systeme – UB Ex 4

(4-6) Client Server Kommunikation

Client schickt Druckauftrag:
Server sendet Bestätigung (eingetroffen)
Bei Absturz sendet Server „wieder da" an alle Clients
Server sendet auch Fertig MSG. Einmal angestoßener Auftrag wird stets ausgeführt

Fertig: 1) bevor der Druckauftrag angestoßen wird oder
2) nachdem der Druckauftrag angestoßen worden ist.

Nach Absturz des Servers ~~sendet~~ retry an Client:
1) immer 2) nie
3) nur, wenn Fertig empfangen wurde
4) nur, wenn Fertig nicht empfangen wurde

P = Druckauftrag angestoßen 0: Auftrag nicht ausgeführt
F = Fertig 1: genau einmal
C = Absturz 2: doppelt

(PF) PC, PFC, C(FP)

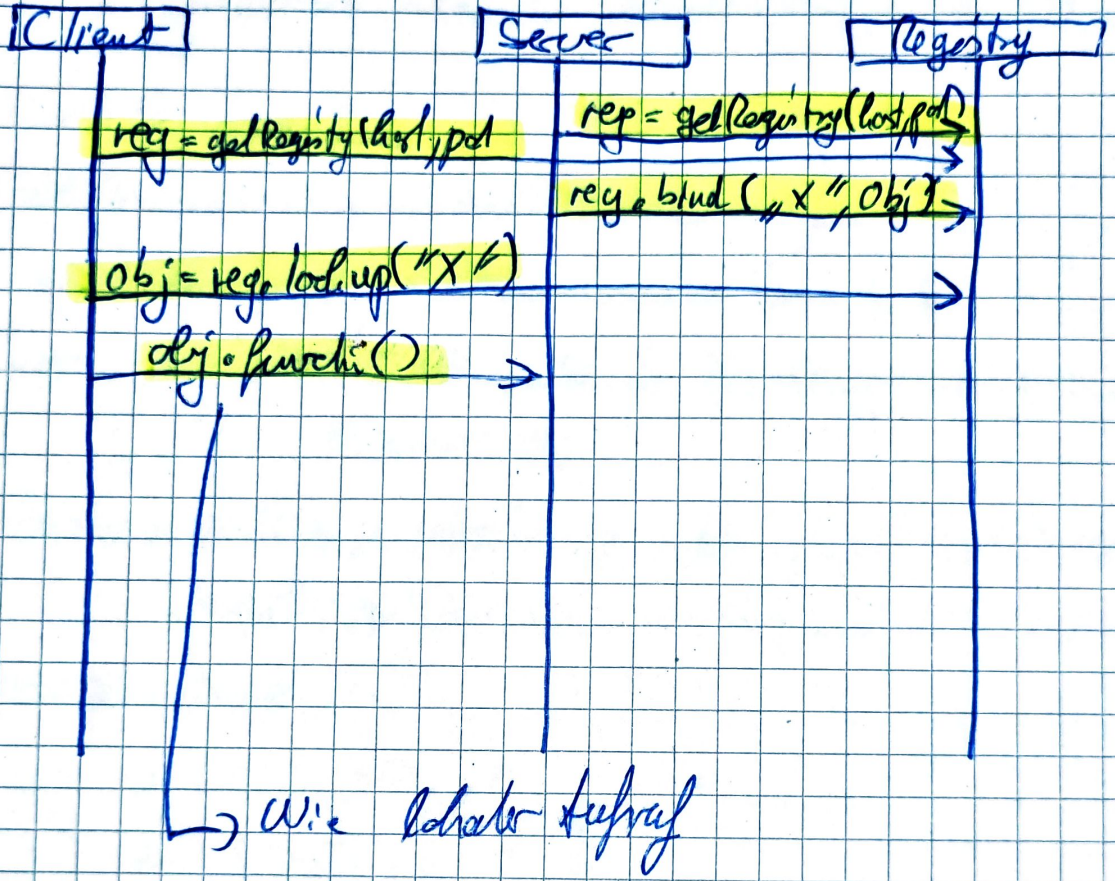
(FP) FC, FPC, CPF

	PFC	PC(F)	C(PF)	C(FP)	F(CP)	FPC
a	2	2	1	1	1	2
b	1	1	0	0	0	1
c	2	1	0	0	1	2
d	1	2	1	1	0	1
	P F			F P		

VL - KVB - Ex 4

(4-7) RMI - Registry

Java RMI verwendet eine Registry zur Registrierung von Objekten



→ Nicht alle Objekte müssen Registriert werden.
Methodenaufruf kann auch Referenz auf Objekt zurück geben :)